Project 3 — DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely

Title: DocuMind Smart Document Agent Ask Your Documents Anything Safely

Type: Multi-agent (3 agents) · **Stack:** Python, LangGraph, RAG (Chroma), LangChain · **Level:** Intermediate

Tagline: A 3-agent LangGraph system — Retriever, Reasoner, and Approval — that grounds every answer in your documents and pauses for a human before anything risky goes out.

> This is the **reading guide** — the story, the folder structure and why each part exists, setup, the 5 build steps, and which document to read for each idea. The other file, README.md, is the **repo README** — you copy it into your repo folder at the end and push it to GitHub. You write every folder, file and line of code yourself; the only thing already created for you is the empty repo folder `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/` .

Charter (what this project is)

A LangGraph app with three agents in one graph: a **Retriever** (searches your documents and judges whether what it found is good enough), a **Reasoner** (answers only from what was found, citing each file), and an **Approval** agent (approves safe drafts by itself, and pauses for a human with `interrupt()` only when a check fails). This is where "an agent" becomes "a graph" — state you can see, save, pause and resume.

Jump to: Read first · Project structure · Setup · How testing works · Step 1 · Step 2 · Step 3 · Step 4 · Step 5 · Checklist · Publish

The Story — what you're actually building

Imagine a support agent asked about *your own* documents — a refund policy, a warranty, a price list — not something the model learned in training. It must answer only from what's actually in those documents. And when it's asked to *do* something ("send this to the customer"), a person should get a say first — but not on every single question, or nobody would use it.

That's what you're building. The **Retriever** goes and looks: it searches your documents and honestly judges whether the best match is close enough to answer from, instead of handing back "the closest thing" no matter how far off. The **Reasoner** writes the answer from those excerpts only, and names

the file behind each fact. The **Approval** agent is the last check: it runs a small checklist, approves safe drafts by itself, and pauses for a person only when a check fails.

Underneath, it's all one **graph**, not a loop hidden in a function. A graph can be drawn, its state saved to disk, paused mid-run, and resumed later — even after a restart.

The steps build it in order: Step 1 proves the graph mechanics. Step 2 adds saved state and a human pause. Step 3 adds the Retriever and its relevance bar. Step 4 adds the Reasoner. Step 5 swaps the always-ask pause for the Approval agent.

What you're building, in one line: a 3-agent LangGraph pipeline — search and judge, answer from the found text, approve or pause for a human.

Why this needs to exist: a model answering from training data can sound sure about your documents and be completely wrong, and letting an agent act with no human check is risky.

When you'd reach for this at a real job: answers must come from your own private documents, and some requests lead to actions that need a person's sign-off — but not every run.

How it works, mechanically: a `StateGraph` carries one typed state through the nodes. Conditional edges pick the path (search or not; grounded or not). `interrupt()` pauses the graph, a SQLite checkpointer saves the state, and `Command(resume=...)` continues it.

Why not paste all the documents into the prompt? Real document sets are far bigger than a context window, and most of the text has nothing to do with the question. **Why not ask a human every time, or never?** Always-asking doesn't scale; never-asking lets a wrong, risky action through.

Where This Fits in the 5-Project Arc

```
Project 1               → 2 agents, Python router, raw SDK
Project 2               → 2 agents, hand-built agent loop, tools
Project 3 (you are here) → 3 agents, LangGraph, RAG + human approval
                           (Retriever + Reasoner + Approval)
Project 4               → 4-5 agents, one Supervisor directs the rest
Project 5               → Project 4, made production-ready
```

Project 3's new ideas: a **graph** (saved, pausable state), **retrieval** from your own documents, and a **human in the loop** only when needed. There's still no supervisor — the routing is fixed conditional edges. Dynamic routing is Project 4's job.

What to Read First (docs this project uses)

You need to understand…	Read this	Used in
`load_config()`, `get_logger()`	Doc01 Build Task	Step 1
`prompt | model | parser`	Doc05 — LCEL basics	Steps 1, 4
`StateGraph`, state, nodes, edges	Doc09 — first graph	Step 1
`Annotated[list, operator.add]`	Doc09 Build Task	Step 1
Checkpointers, `interrupt()`, `Command(resume=...)`	Doc09 — interrupt and resume	Step 2
`SqliteSaver`, resuming in a new process	Doc09 — interrupt and resume	Step 2
Chunking, embeddings, Chroma, `retrieve()`	Doc08 Build Task	Step 3
Distance scores (lower = closer)	Doc08 — similarity	Step 3
Conditional edges	Doc09 — conditional routing	Step 3
Search only when needed	Doc10 — conditional search	Step 3
"Search found nothing"	Doc10 — empty search	Step 3
An approval pause	Doc10 — approval pause	Steps 2, 5
The whole app, as a practice run	Doc10 Build Task	All steps

Reused code, unchanged except for its folder and import lines: `config.py`, `exceptions.py`, `utils/logger.py` (Doc01), `utils/prompts.py` (Project 1), and `rag/chunking.py`, `rag/ingest.py`, `rag/retriever.py` (Doc08 — the OpenAI client moved inside each function). The graph patterns — `build_graph(checkpointer)`, `path` with `operator.add`, `interrupt()` + `Command(resume=...)`, `SqliteSaver` — are Doc09's, and the node names follow Doc10.

Project Structure (the professional layout)

Same rule as before: only folders with a real job. Project 3 adds `rag/` (the document search), `data/docs/` (the documents), and puts the graph's own files (`state.py`, `nodes.py`, `graph.py`) at the top of the package.

Names: the repo folder vs. the Python package

- **Repo folder:** `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/` — already created, and the exact name of the GitHub repo.

- **Python package:** `src/`, right next to `main.py`. It holds all the app's code, and every file imports from it the same way: `from src.graph import build_graph`.
- **Files:** `snake_case.py`, one job each; test files start with `test_`; agent files end in `_agent.py`.
- **Documents:** plain `.md` files in `data/docs/`, named for their topic (`refund_policy.md`) — the Reasoner cites these names.

The final layout (after Step 5)

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── data/
│   └── docs/               the documents DocuMind answers from
│       ├── account_security.md
│       ├── pricing.md
│       ├── refund_policy.md
│       ├── shipping.md
│       └── warranty.md
├── scripts/
│   ├── pause_demo.py       part 1: start a run, stop at the pause
│   └── resume_demo.py      part 2: a new process finishes it
├── src/
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── approval_agent.py  checklist + interrupt()
│   │   ├── reasoner_agent.py  grounded, cited answer
│   │   └── retriever_agent.py route, search, judge relevance
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py          get_model() -> ChatOpenAI
│   ├── prompts/
│   │   ├── general.txt     for non-document questions
│   │   └── reasoner.txt    "only the excerpts, cite the file"
│   ├── rag/
│   │   ├── __init__.py
│   │   ├── chunking.py     chunk_by_chars/paragraph     (Doc08)
│   │   ├── ingest.py       build_vector_store()         (Doc08)
│   │   ├── retriever.py    retrieve() -> list[dict]     (Doc08)
│   │   └── store.py        get_collection() — built once
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py       get_logger()                 (Doc01)
│   │   └── prompts.py      load_prompt(name)        (Project 1)
│   ├── __init__.py
│   ├── config.py           Config, load_config()        (Doc01)
│   ├── exceptions.py       MissingConfigError           (Doc01)
│   ├── graph.py            build_graph(checkpointer)    (Doc09)
```

```
│   ├── nodes.py            clean_task, answer_node
│   └── state.py            DocState, start_state()      (Doc09)
├── tests/
│   ├── integration/
│   │   └── test_live_graph.py      real API calls
│   └── unit/
│       ├── test_approval_checks.py no API key needed
│       ├── test_chunking.py        no API key needed
│       ├── test_config.py          no API key needed
│       ├── test_nodes.py           no API key needed
│       └── test_routing.py         no API key needed
├── .env                    your real key — never committed
├── .env.example            the setting names, no real values
├── .gitignore              keeps .env, .venv/, checkpoints.sqlite out
├── LICENSE                 lets others reuse your code (MIT)
├── main.py                 the terminal loop — the file you run
├── README.md               the repo's front page (copied in at the end)
└── requirements.txt        the libraries, for pip install -r
```

Why each folder exists

- **`data/docs/`** — your documents. **Why:** they're *data*, not code — replace them with your own and nothing else changes.
- **`scripts/`** — the two-part pause/resume demo. **Why:** proving a resume from disk needs two separate processes — that's two scripts, not a test function.
- **`src/`** — the package, right next to `main.py`: every other `.py` file the app is made of. The three graph files sit at its top level: `state.py` (what the graph carries), `nodes.py` (small non-agent nodes), `graph.py` (how they connect). **Why:** these three are what you read first to understand the app — Doc09's own file names.
- **`agents/`** — one file per agent: Retriever, Reasoner, Approval. **Why:** three agents, three jobs, three files — you can point at each one.
- **`models/`** — builds the chat model. **Why:** every node gets the same model and settings; changing the model is one line.
- **`prompts/`** — prompt text as `.txt` files. **Why:** the Reasoner's rules are the most-edited text in the project; they shouldn't be buried in code.
- **`rag/`** — everything about searching documents: splitting, embedding and storing, searching. **Why:** retrieval is its own subject (Doc08) and works without the graph; keeping it apart means you can test and tune search alone.
- **`utils/`** — the logger and the prompt loader, used everywhere.
- **`tests/integration/`** — the whole graph against the real API, every path. **Why:** costs money, so it's run on purpose.
- **`tests/unit/`** — the config, the small nodes, chunking, routing, relevance and the approval checklist, with hand-made inputs. **Why:** fast, free, no key — run after every change.

Why each root file exists

- `.env` — your real API key; never committed.
- `.env.example` — which settings to create, with no real values.
- `.gitignore` — stops `.env`, the venv, cache files and `checkpoints.sqlite` (the saved graph states) from being pushed.
- `LICENSE` — the MIT license text, created in Setup. Without one, nobody may legally reuse your code.
- `main.py` — the file you run: the question loop and the approval conversation.
- `README.md` — what someone sees first on GitHub.
- `requirements.txt` — the library list for `pip install -r requirements.txt`.

Why each file exists (what and why)

Every file you'll write, in one place — the same What/Why list the documents give after each tree. The steps below build these files one at a time; come back here whenever you forget why a file exists.

- `data/docs/*.md`
  - **What:** Short Markdown files, a few paragraphs each, with no heading lines.
  - **Why:** These are the only things DocuMind answers from — replace them with your own documents.
- `scripts/pause_demo.py`
  - **What:** One `main()` that opens the SQLite checkpointer, runs a task that needs approval on a fixed thread id (`demo-1`), prints the paused draft, and exits.
  - **Why:** It is part 1 of proving that a paused run survives the program closing.
- `scripts/resume_demo.py`
  - **What:** One `main()` that opens the same SQLite file, reads `graph.get_state(config)`, and — if the run is still paused — asks yes/no and resumes it with `Command(resume=...)`.
  - **Why:** It is part 2: a brand-new process finishes the run, which proves the state really came from disk.
- `agents/approval_agent.py`
  - **What:** The node `approval_node(state)`. From Step 2 it pauses with `interrupt({answer, failed_checks})` and approves only if the person answers "yes". Step 5 adds `ACTION_WORDS` and three check functions (`cites_a_source()`, `asks_for_an_action()`, `run_checks()`), and the node first runs `run_checks()` — if nothing failed, it approves by itself.
  - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `agents/reasoner_agent.py`

  - **What:** The node `reason_node(state)`, which returns the `answer`. In Step 3 it simply quotes the best excerpt with its file name; in Step 4 it keeps its name and output, and a model chain writes the answer instead, with the new helper `format_sources(sources)` putting each excerpt's file name in front of it.
  - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.
- `agents/retriever_agent.py`
  - **What:** `SEARCH_KEYWORDS`, `MAX_DISTANCE`, and five functions: `route_search(state)` → "search" or "skip"; `is_relevant(sources)` compares the smallest score with `MAX_DISTANCE`; `retrieve_node(state)` searches and stores `sources` and `grounded`; `has_grounding(state)` → "reason" or "no_grounding"; `no_grounding_node(state)` returns an honest "not found" answer.
  - **Why:** A vector store always returns something. Only a written relevance bar decides whether it's good enough — otherwise the answer would be built on nothing.
- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
  - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `prompts/*.txt`
  - **What:** Two text files: the general prompt (with `{task}`) and the Reasoner's rules — only the excerpts, cite the file in brackets, and an exact sentence when the documents don't cover it (with `{sources}` and `{task}`).
  - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.
- `rag/chunking.py`
  - **What:** Two functions from Doc08: `chunk_by_chars(text, chunk_size)` cuts the text every N characters; `chunk_by_paragraph(text)` splits on blank lines and drops empty pieces.
  - **Why:** Search works on small pieces, so it can find the right paragraph instead of a whole file.
- `rag/ingest.py`
  - **What:** `build_vector_store(doc_folder, chunk_fn, collection_name)`: delete and recreate the Chroma collection, chunk every `.md` / `.txt` file, embed all chunks in **one** `embeddings.create()` call, and `collection.add()` them with ids and metadata.
  - **Why:** It builds the searchable store from `data/docs/`. Deleting first means running it twice never duplicates chunks.
- `rag/retriever.py`
  - **What:** `retrieve(collection, query, k=3)`: return `[]` for an empty store; otherwise embed the query, `collection.query(...)`, and return a list of dicts with `text`, `source_file`, `chunk_index` and `score` (a distance — lower is closer).

    - **Why:** This is the search itself, returning a checked, simple shape instead of the store's raw output.
- `rag/store.py`
    - **What:** A `StoreHolder` class with a `collection` field, one `store` object, and `get_collection()`, which builds the store only when `store.collection` is still `None`.
    - **Why:** The documents are embedded once per run — not for every question, and not at import time before the API key is loaded.
- `utils/logger.py`
    - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
    - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`
    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `__init__.py` (every code folder)
    - **What:** An empty file. You create one in every folder that holds Python code — nothing goes inside it.
    - **Why:** Python only treats a folder as a package when this file is there. Without it, an import like `from <package>.<folder> import ...` fails with `ModuleNotFoundError`.
- `config.py`
    - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
    - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
    - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `graph.py`
    - **What:** One function, `build_graph(checkpointer=None)`. It uses a `MemorySaver` if none is given, adds the six nodes, adds `START → clean_task`, two `add_conditional_edges`

(search or skip; grounded or not), plain edges to `END`, and returns `compile(checkpointer=checkpointer)`.
    - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `nodes.py`
    - **What:** Two node functions: `clean_task(state)` returns the stripped task and `["clean_task"]` for the path; `answer_node(state)` runs a general-knowledge chain and returns the answer, labelled as general knowledge.
    - **Why:** One tidies the question once for every later node; the other answers questions that aren't about your documents — clearly labelled as such.
- `state.py`
    - **What:** A `DocState` TypedDict (`task`, `path` with `Annotated[list[str], operator.add]`, `sources`, `grounded`, `approved`, `answer`) and `start_state(task)`, which returns a dict with every field set. Each step only adds fields.
    - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `tests/integration/test_live_graph.py`
    - **What:** A `run(graph, thread_id, task, decision)` helper that resumes a paused run with `decision`, and eight test functions against the real API, each printing `OK` or `WRONG`: a general question; an action task that pauses, is approved, is rejected; a grounded document answer; an unknown topic; a cited answer; and a safe answer that is auto-approved.
    - **Why:** It proves every path through the graph works with the real model, including both human decisions.
- `tests/unit/test_approval_checks.py`
    - **What:** Three test functions on hand-made states, each printing `OK` or `WRONG`: a grounded, cited answer passes every check (`[]`); an answer with no citation fails `cites no source file`; a task asking for an action fails `the task asks for an action`.
    - **Why:** A broken check would send every answer to a human, or none — this catches it in a second, with no API key.
- `tests/unit/test_chunking.py`
    - **What:** Two test functions, each printing `OK` or `WRONG`: 650 characters in chunks of 300 give lengths `[300, 300, 50]`, and `chunk_by_paragraph()` drops the empty pieces between blank lines.
    - **Why:** Search quality starts with the chunks — a chunker bug would quietly make whole paragraphs unfindable.
- `tests/unit/test_config.py`
    - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting

name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
    - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `tests/unit/test_nodes.py`
    - **What:** Two test functions with hand-made input: `clean_task()` strips the task and adds `["clean_task"]` to the path, and `start_state()` starts with an empty `path` and `answer`. Each prints `OK` or `WRONG`.
    - **Why:** A node that forgot its path entry, or a missing starting field, would only show up as a strange run later.
- `tests/unit/test_routing.py`
    - **What:** Seven test functions, one per rule, on hand-made states and score lists: a document question searches, a general one skips; a best score of 0.4 counts, 1.7 doesn't, no sources at all doesn't; grounded goes to the Reasoner, not grounded to the honest "not found".
    - **Why:** A wrong comparison (`>` instead of `<=`) would only show up as strange answers — here it shows up as `WRONG`.
- `main.py`
    - **What:** `main()` loads the config, opens `SqliteSaver.from_conn_string(DB_PATH)`, builds the graph, then loops: read a question, make a unique thread id, `graph.invoke(start_state(task), config)`, and if the result has `"__interrupt__"`, call `ask_human()` and resume with `Command(resume=decision)`. It prints the path and the answer.
    - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.

How imports work in this layout

- Every folder with Python files gets an empty `__init__.py`. It marks the folder as a package, so `src.agents.retriever_agent` is importable. (`prompts/` holds only `.txt` files and `data/` only documents, so they don't need one.)
- Python looks for imports in the folder you **run from**. `main.py` and `src/` sit side by side in the repo folder, so `python main.py` finds `from src.graph import build_graph` right there.
- Tests and scripts live in sub-folders, so run them as modules from the repo folder: `python -m tests.unit.test_routing`. The `-m` tells Python to start from the folder you're in — the repo folder — so `src` is found there too.
- Always run commands from the **repo folder**. `data/docs/`, the prompts folder and `checkpoints.sqlite` are all found from there.

Left out on purpose (and when they come)

Folder / file in a big template	Why not here	Where it appears
`api/`, `routes.py`	A terminal app, no web server	Project 5 (Doc12)
`supervisor.py`	Routing is fixed conditional edges	Project 4 (Doc11)
`vectorstore/` on disk	5 small documents re-embed in a second at startup	When the document set grows
`evaluation/`	The tests check paths; answer quality scoring is later	Doc13
`Dockerfile`, CI	Nothing to deploy yet	Doc12-13
pytest, `conftest.py`	Plain test scripts, same as Projects 1-2	Doc13
A frontend	The approval question works in the terminal	Streamlit, when a project needs one

How the structure grows, step by step

Step	New files	Changed files
Setup	root files, `src/__init__.py`	—
1	`main.py`, `config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py`, `models/llm.py`, `prompts/general.txt`, `state.py`, `nodes.py`, `graph.py`, `tests/unit/test_config.py`, `tests/unit/test_nodes.py`, `tests/integration/test_live_graph.py`	—
2	`agents/approval_agent.py`, `scripts/pause_demo.py`, `scripts/resume_demo.py`	`main.py`, `state.py`, `graph.py`, `test_live_graph.py`
3	`rag/` (4 files), `data/docs/`, `agents/retriever_agent.py`, `agents/reasoner_agent.py`, `tests/unit/test_chunking.py`, `tests/unit/test_routing.py`	`state.py`, `graph.py`, `test_live_graph.py`
4	`prompts/reasoner.txt`	`agents/reasoner_agent.py`, `test_live_graph.py`
5	`tests/unit/test_approval_checks.py`	`agents/approval_agent.py`, `test_live_graph.py`

Setup (do this once, before Step 1)

Read first: Doc01 — Virtual environments, Doc01 — `.env` files. The same steps as Projects 1 and 2, with a different library list.

1. Go into the repo folder, start git, and make a venv. From the curriculum's root folder (`multi-agent-ai-engineering-curriculum/`):

```
cd project_3_documind_rag_agent
cd DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely
git init
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

2. Create `.gitignore` **before** your first commit:

```
# secrets — never pushed
.env
# the virtual environment — rebuilt from requirements.txt
.venv/
# Python's compiled cache files
__pycache__/
*.pyc
# saved graph states (Step 2) — local only
checkpoints.sqlite
```

3. Create `.env` and `.env.example` , the same as Projects 1 and 2:

```
# .env — never commit this file
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

```
# .env.example — commit this one
# Copy this file to .env and fill in the real values. Never commit .env.
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

4. Create `requirements.txt` :

```
chromadb>=1.0
langchain-core>=1.0
langchain-openai>=1.0
langgraph>=1.0
```

```
langgraph-checkpoint-sqlite>=2.0
python-dotenv>=1.0
```

New since Project 2: `langgraph` (the graph), `langgraph-checkpoint-sqlite` (saved state on disk), `chromadb` (the local vector store from Doc08). `langchain-openai` brings `openai`.

5. Create an empty `src/__init__.py`, then install the libraries:

```
pip install -r requirements.txt
```

Check it worked: from the repo folder, `python -c "import src; print('ok')"` prints `ok`. It works because the `src/` folder sits right there — that's why every command runs from the repo folder.

Your documents: Step 3 needs 5-10 short documents in `data/docs/` — your own notes, policies or wiki pages as `.md` files, a few paragraphs each, **no heading lines**. The solution's example set is a small company's refund, shipping, warranty, account-security and pricing policies. For a bigger public set, the Hugging Face dataset `rag-datasets/rag-mini-wikipedia` is small and made for practicing RAG.

6. Create `LICENSE` — the MIT license, so others may reuse your code. Put the current year and your own name on the Copyright line:

```
MIT License

Copyright (c) 2026 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Why This Project Is Good for Your Portfolio

- **What problem it solves:** an agent whose answers come from your documents, whose every step is visible in a `path` log, and which pauses for a person before anything risky — then resumes, even after a restart.
- **Why it matters:** typed state, branching, saved state and human approval are exactly what "how would you make this safe to run for real?" is asking about.
- **When you'd build something like this at a job:** document Q&A, support answers from a policy base, anything that must survive a restart or get sign-off before acting.
- **How it's built:** a `StateGraph` with typed state, conditional search, a written relevance bar, a citing Reasoner, a checklist-based Approval agent, and a SQLite checkpointer.

Problems you'll likely run into, and how to fix them:

Problem	Fix
Every question counts as "grounded"	A vector store always returns its k closest chunks — judge the **best score** against a written bar (`MAX_DISTANCE`), not "was anything returned"
Off-topic questions match a heading	A heading line alone becomes its own chunk — write documents as paragraphs, no heading lines
The Reasoner adds facts from memory	Tell it: only the excerpts, cite the file, and an exact sentence for "not covered"
A resumed run searches again, or loses its sources	Keep `sources` in the state — the checkpoint saves them
Resuming works in one script but not after a restart	`MemorySaver` lives in memory; use `SqliteSaver` and test with two separate processes
A log line prints twice after resuming	A node with `interrupt()` runs again from its first line on resume — keep real actions **after** the `interrupt()` call
`KeyError` in a node	A field wasn't in the starting state — always start runs with `start_state(task)`

Built During These Documents

08_rag → Async Python gate → 09_langgraph → 10_agent_workflows (on top of Docs 01-07)

Plan Before You Code

See 15_five_projects_index: write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks, and **draw the graph** before writing any

node.

How To Build This — Step by Step

The project grows step by step, and **after every step it runs**: Step 1 answers through a 2-node graph, Step 2 pauses every answer for you, Step 3 answers from your documents, Step 4 writes cited answers, Step 5 asks you only when needed. Build it yourself first; use the hints if you're stuck, and the solution only to compare.

Every command runs from the **repo root** with the venv active.

How testing works in this project

Story: DocuMind's promise is "it answers only from your documents, and asks before anything risky goes out" — two promises a casual try can't confirm. So every step adds the tests for the path it adds: the general answer, the pause, the grounded answer, the honest "not found", the citation, the auto-approval. When something breaks, the newest `WRONG` points straight at the newest code. **If not:** a prompt change could quietly start answering from nothing, or approving actions, and you'd only find out from a user.

- **Two kinds of test:**
  - `tests/unit/` — code that needs no model: settings, the small nodes, the chunkers, every routing and relevance rule, the approval checklist. No API key, no cost, under a second. **When:** after every change to the file it tests — run them all the time.
  - `tests/integration/test_live_graph.py` — the whole graph against the real API, one case per path. Real API calls (a fraction of a cent). **When:** at the end of every step, and after changing a prompt, `MAX_DISTANCE`, `SEARCH_KEYWORDS` or `ACTION_WORDS`.
- **One pattern for every case** — so the output never needs judging:

```
def test_something() -> None:
    result = "OK"
    if <it came out wrong>:
        result = "WRONG"
    print(f"case 1 (what it checks) -> {result}")
```

- **An error you expect** (like `MissingConfigError`): start with `result = "WRONG"`, and set `result = "OK"` inside `except TheError:`.
- **One function per case**, all called at the bottom under `if __name__ == "__main__":`. Each file numbers its own cases, and case numbers never change once written.
- **Model answers vary**, so check one fact — a node name in `path`, a file name in brackets, `grounded` is `True` — never the exact wording.
- **Each live case gets its own `thread_id`** (`"t1"`, `"t2"`, ...), so saved states never mix.

- **Run from the repo root with `-m`:** `python -m tests.unit.test_routing` (dots, no `.py`). Running the file by its path fails with `No module named 'src'`, because Python then starts looking for imports in `tests/unit/`.
- **Got a `WRONG`?** Read that one test function, then the code it calls. Fix the code — change the test only if the test itself checks the wrong thing.

Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 1 of 5

- **Where you are now:** Setup is done — nothing runs yet.
- **What's new in this step:** the shared files (config, errors, logger, prompt loader, model), and the three graph files: `state.py`, `nodes.py` (`clean_task` → `answer_node`), `graph.py`, plus `main.py`. Tests: `test_config.py`, `test_nodes.py`, and `test_live_graph.py` with case 1.
- **Why:** every later step adds nodes and branches to this one graph. Prove state really flows from node to node before anything gets harder to read.
- **How:** `StateGraph(DocState)` → `add_node` × 2 → `add_edge(START, ...)`, `add_edge(..., END)` → `compile()` → `graph.invoke(start_state(task))`.
- **Working after this step:** `python main.py` answers questions and prints `path: clean_task -> answer_node`, and all three test files print `OK`.

Read first: Doc09 — first graph, Doc09 Build Task.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Bring in the shared files: `config.py`, `exceptions.py`, `utils/logger.py`, `utils/prompts.py` (folder `src/prompts`), and `models/llm.py` with a no-argument `get_model()`.
2. Write `state.py`: a `TypedDict` with `task`, `path: Annotated[list[str], operator.add]` and `answer`, plus `start_state(task)`.
3. Write `nodes.py`: `clean_task` (strips the task) and `answer_node` (a Doc05 chain with `prompts/general.txt`). Each returns only the fields it changes, plus its own name in `path`.
4. Write `graph.py` (`build_graph()`) and `main.py` (a question loop that prints `path` and `answer`).
5. Trace the state by hand after each node, then run it and compare.
6. Write the first tests — `tests/unit/test_config.py`, `tests/unit/test_nodes.py`, and `tests/integration/test_live_graph.py` with case 1 — and run all three with `python -m`.

The root files — `.gitignore`, `.env`, `.env.example`, `requirements.txt` and `LICENSE` — come from Setup; this step adds the files below.

Your files after Step 1:

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── src/
│   ├── models/
│   │   ├── __init__.py  (new)
│   │   └── llm.py       (new) get_model()
│   ├── prompts/
│   │   └── general.txt  (new)
│   ├── utils/
│   │   ├── __init__.py  (new)
│   │   ├── logger.py    (new)
│   │   └── prompts.py   (new)
│   ├── __init__.py
│   ├── config.py        (new)
│   ├── exceptions.py    (new)
│   ├── graph.py         (new) build_graph(): 2 nodes, plain edges
│   ├── nodes.py         (new) clean_task, answer_node
│   └── state.py         (new) DocState: task, path, answer
├── tests/
│   ├── integration/
│   │   └── test_live_graph.py (new) case 1
│   └── unit/
│       ├── test_config.py     (new) no API key needed
│       └── test_nodes.py      (new) no API key needed
└── main.py              (new) question loop -> graph
```

Why these files (what and why):

- `models/llm.py`
  - **What:** A `MODEL_NAME` constant and one function, `get_model()`, which returns `ChatOpenAI(model=MODEL_NAME, temperature=0)`.
  - **Why:** Every chain and agent gets the same model and settings from here. To try another model you change one line, not every file.
- `prompts/general.txt`
  - **What:** This text file holds the general prompt. The code loads it with `load_prompt("general")`.
  - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.
- `utils/logger.py`
  - **What:** One function, `get_logger(name)`. It gets the logger for that name and, only the first time (`if not logger.handlers:`), adds a console handler with a timestamp format and the level from `LOG_LEVEL`.
  - **Why:** Every file logs in the same format with one line of code. Without the `if not logger.handlers:` guard, every log line would print twice.
- `utils/prompts.py`

    - **What:** One constant, `PROMPTS_DIR` (the prompts folder, from the repo root), and one function, `load_prompt(name)`, which opens `PROMPTS_DIR + "/" + name + ".txt"` and returns its text.
    - **Why:** Prompts live in text files, so you can change how an agent behaves without editing Python — and no agent has to open files itself.
- `config.py`
    - **What:** A `Config` dataclass with two fields (`openai_api_key`, `log_level`), a helper `require_env(key)` that raises `MissingConfigError` when a setting is empty, and `load_config()`, which calls `load_dotenv()`, reads both settings and returns a `Config`.
    - **Why:** Every file gets its settings from one place. If the key is missing, the app stops at startup with one clear line — without this file, the missing key would show up later as a confusing API error.
- `exceptions.py`
    - **What:** A few small classes that inherit from `Exception` — at least `MissingConfigError`. Each one is just `class Name(Exception): pass`, with a docstring saying when it's raised.
    - **Why:** Code can catch one specific problem (`except MissingConfigError:`). Without named errors you'd catch every `Exception`, and real bugs would be hidden along with the expected problem.
- `graph.py`
    - **What:** One function, `build_graph()`: two nodes in a straight line — `START → clean_task → answer_node → END` — compiled into a graph.
    - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `nodes.py`
    - **What:** Two node functions: `clean_task(state)` returns the stripped task and `["clean_task"]` for the path; `answer_node(state)` runs a general-knowledge chain and returns the answer, labelled as general knowledge.
    - **Why:** One tidies the question once for every later node; the other answers questions that aren't about your documents — clearly labelled as such.
- `state.py`
    - **What:** A `DocState` TypedDict with `task`, `path` (`Annotated[list[str], operator.add]`, so each node adds its name) and `answer`, and `start_state(task)`, which sets every field.
    - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `test_live_graph.py`
    - **What:** A `run(graph, thread_id, task)` helper and one test function against the real API: a general question goes through both nodes and the answer mentions Paris (case 1).
    - **Why:** It proves every path through the graph works with the real model, including both human decisions.

- `test_config.py`
  - **What:** Three test functions, no API call, each printing `OK` or `WRONG`: `require_env()` raises `MissingConfigError` for a missing and for an empty setting (a made-up setting name, so your `.env` is never touched), and `load_config()` returns a `Config` when a key is set.
  - **Why:** It proves a missing key stops the app with one clear error, with no API call — the first thing that must work.
- `test_nodes.py`
  - **What:** Two test functions with hand-made input: `clean_task()` strips the task and adds `["clean_task"]` to the path, and `start_state()` starts with an empty `path` and `answer`. Each prints `OK` or `WRONG`.
  - **Why:** A node that forgot its path entry, or a missing starting field, would only show up as a strange run later.
- `main.py`
  - **What:** One function, `main()`: it loads the config, builds the graph with `build_graph()`, then loops — read a question, `graph.invoke(start_state(task))`, print the `path` and the `answer`.
  - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (missing setting)** — `tests/unit/test_config.py`
  - **Check:** with a made-up setting name removed from `os.environ`, `require_env()` raises `MissingConfigError`.
  - **Why:** a missing key must stop the app at startup with one clear line.
  - **When:** after any change to `config.py` or `exceptions.py`.
  - **If not:** a missing key would surface later as a confusing `AuthenticationError` from OpenAI.
- **case 2 (empty setting)** — `tests/unit/test_config.py`
  - **Check:** the same name set to `""` also raises `MissingConfigError`.
  - **Why:** `OPENAI_API_KEY=` with nothing after it is "set" but never usable.
  - **When:** same as case 1.
  - **If not:** the app would start happily and fail on the first real call.
- **case 3 (config loads)** — `tests/unit/test_config.py`
  - **Check:** with `OPENAI_API_KEY` set in `os.environ`, `load_config()` returns a `Config` holding that key.
  - **Why:** cases 1–2 only prove the *error* path; this proves the values actually come back.
  - **When:** same as case 1.

    - **If not:** a missing `return value` in `require_env()` passes cases 1–2, and every setting silently becomes `None`.
- **case 1 (clean_task tidies the task)** — `tests/unit/test_nodes.py`
    - **Check:** `clean_task({"task": " What is a refund? "})` returns the stripped task and `["clean_task"]` for the path.
    - **Why:** every later node — and the keyword routing in Step 3 — reads this cleaned task.
    - **When:** after any change to `nodes.py`.
    - **If not:** a node that forgot its `path` entry would make the printed path lie about what ran.
- **case 2 (start_state starts empty)** — `tests/unit/test_nodes.py`
    - **Check:** `start_state("hello")` has the task, an empty `path` and an empty `answer`.
    - **Why:** every node can rely on every field existing from the first step.
    - **When:** after any change to `state.py`.
    - **If not:** a missing field would crash a node with a `KeyError` halfway through a run.
- **case 1 (general question)** — `tests/integration/test_live_graph.py`
    - **Check:** "What is the capital of France?" goes through `answer_node`, and the answer mentions Paris.
    - **Why:** proves the key, the model, the prompt file, the state and the graph wiring work together.
    - **When:** at the end of the step, and after changing `graph.py`, `nodes.py` or `prompts/general.txt`.
    - **If not:** a broken edge or prompt variable would be found by the first user, not by you.

Step 2 — Saved State and a Human Pause Before Anything Goes Out

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 2 of 5

- **Where you are now:** after Step 1 a 2-node graph answers questions — but nothing is saved, and every answer goes straight out.
- **What's new in this step:** the Approval agent's file, `agents/approval_agent.py`, with `approval_node` — for now it asks a person about **every** answer, using `interrupt()`. Plus a SQLite checkpointer passed into `build_graph()`, a thread id per question, `ask_human()` in `main.py`, and the two-script pause/resume demo.
- **What stays the same:** `clean_task` and `answer_node` (they never change again), the model and the prompt.
- **Why:** a graph can stop mid-run, save everything, and continue later — even after a restart. That's what lets a person approve an answer before it goes out.
- **How:** `interrupt({"answer": ..., "failed_checks": [...]})` pauses and saves; the app sees `"__interrupt__"` in the result, asks yes/no, and resumes with `graph.invoke(Command(resume=decision), config)`.

- **Working after this step:** every answer waits for your yes/no; a paused run can be finished by a different process; live cases 1-4 print `OK`.

Read first: Doc09 — interrupt and resume, Doc10 — approval pause.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Add `approved` to `state.py` (and to `start_state`).
2. Create `agents/approval_agent.py` (with an empty `agents/__init__.py`) and write `approval_node`: a fixed `reasons` list ("every answer is checked by a person for now"), `interrupt({"answer": ..., "failed_checks": reasons})`, then `"yes"` → approved, anything else → the answer becomes "Not approved — nothing was sent.".
3. Change `build_graph(checkpointer=None)` to take a checkpointer (a `MemorySaver` if none is given), and add `approval_node` after `answer_node`.
4. In `main.py`, open `SqliteSaver.from_conn_string("checkpoints.sqlite")`, make a thread id per question, and handle the pause with `ask_human()` (it prints the answer and the reasons).
5. Write `scripts/pause_demo.py` and `scripts/resume_demo.py`, run them one after the other, and check the second one finishes the run.
6. Give the test's `run()` an optional `decision`, build the test graph with `MemorySaver()`, and add cases 2-4: an action task pauses, "yes" approves it, "no" stops it.

Your files after Step 2:

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── scripts/
│   ├── pause_demo.py      (new) part 1: stop at the pause
│   └── resume_demo.py     (new) part 2: a new process resumes it
├── src/
│   ├── agents/
│   │   ├── __init__.py    (new)
│   │   └── approval_agent.py (new) approval_node — asks about everything
│   ├── (Step 1's other files, unchanged)
│   ├── graph.py           (changed) build_graph(checkpointer), + approval
│   └── state.py           (changed) + approved
├── tests/
│   └── integration/
│       └── test_live_graph.py (changed) + cases 2-4, run(decision)
└── main.py                (changed) SqliteSaver, thread ids, ask_human()
```

Why these files (what and why):

- `scripts/pause_demo.py`

    - **What:** One `main()` that opens the SQLite checkpointer, runs a task that needs approval on a fixed thread id (`demo-1`), prints the paused draft, and exits.
    - **Why:** It is part 1 of proving that a paused run survives the program closing.
- `scripts/resume_demo.py`
    - **What:** One `main()` that opens the same SQLite file, reads `graph.get_state(config)`, and — if the run is still paused — asks yes/no and resumes it with `Command(resume=...)`.
    - **Why:** It is part 2: a brand-new process finishes the run, which proves the state really came from disk.
- `agents/approval_agent.py`
    - **What:** One node, `approval_node(state)`: a fixed reason ("every answer is checked by a person for now"), `interrupt({answer, failed_checks})`, then `"yes"` approves and anything else replaces the answer with "Not approved — nothing was sent."
    - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `graph.py`
    - **What:** `build_graph(checkpointer=None)` now takes a checkpointer (a `MemorySaver` if none is given) and adds `approval_node` after `answer_node`.
    - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `state.py`
    - **What:** Step 1's `DocState` plus `approved: bool`, set to `False` in `start_state()`.
    - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `test_live_graph.py`
    - **What:** Step 1's case, plus an optional `decision` on `run()` (resume a paused run with it; `None` leaves it paused), a fresh `MemorySaver`, and cases 2-4: an action task pauses, "yes" approves it, "no" rejects it.
    - **Why:** It proves every path through the graph works with the real model, including both human decisions.
- `main.py`
    - **What:** `main()` now opens `SqliteSaver.from_conn_string(DB_PATH)`, builds the graph with it, gives each question its own thread id, and when the result has `"__interrupt__"`, calls `ask_human()` and resumes with `Command(resume=decision)`.
    - **Why:** One starting point that owns the pause/resume conversation with the user. The graph and agents live in other files.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 2 (pauses)** — `tests/integration/test_live_graph.py`
    - **Check:** an action task ("... Send it to the customer.") run with `decision=None` returns a result with `__interrupt__`.

    - **Why:** the pause is this step's feature — nothing may go out before a person decides.
    - **When:** at the end of the step, and after changing `approval_agent.py` or `graph.py`.
    - **If not:** an answer could go out with nobody checking it — the exact risk this project exists to stop.
- **case 3 (human: yes)** — `tests/integration/test_live_graph.py`
    - **Check:** the same task resumed with `"yes"` ends with `approved=True` and `human_approved` in the path.
    - **Why:** approving must really let the run continue to the end.
    - **When:** same as case 2.
    - **If not:** a resume that never finished would leave every approved answer stuck.
- **case 4 (human: no)** — `tests/integration/test_live_graph.py`
    - **Check:** resumed with `"no"` (in its own thread), the answer becomes "Not approved — nothing was sent."
    - **Why:** a rejection must stop the answer, not just log it.
    - **When:** same as case 2.
    - **If not:** "no" could still send the answer — the worst bug a human-approval step can have. Not a test: `scripts/pause_demo.py` and `scripts/resume_demo.py`. Proving a resume from disk needs two separate processes, so it's two scripts you run by hand, one after the other.

Step 3 — A Retriever Agent That Judges Its Own Search Results

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 3 of 5

- **Where you are now:** after Step 2 answers pause for approval, but they're general guesses — the graph can't read your documents.
- **What's new in this step:** Doc08's retriever (`rag/`), your documents (`data/docs/`), the Retriever agent (route, search, **judge**), an honest "not found" node, the Reasoner's file with a simple `reason_node` that quotes the best excerpt, two conditional edges, and the first tests.
- **What stays the same:** `clean_task`, `answer_node`, `approval_node`, the checkpointer and `main.py`.
- **Why:** a vector store always returns its closest chunks — even when none is relevant. Judging the best score against a written bar is what makes an answer "grounded".
- **How:** `route_search` (keywords) → `retrieve_node` (search + `is_relevant`) → `has_grounding` picks `reason_node` or `no_grounding_node`; `reason_node` → `approval_node`.
- **Working after this step:** document questions are answered with the best excerpt and its file name; off-topic ones get "not found"; general ones skip the search; every unit case and live cases 1-6 print `OK`.

Read first: Doc08 Build Task, Doc10 — conditional search, Doc10 — empty search, Doc09 — conditional routing.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write 5 short documents in `data/docs/` (paragraphs, no heading lines).
2. Bring in Doc08's `chunking.py`, `ingest.py` and `retriever.py` as `rag/`, creating the OpenAI client inside each function. Add `rag/store.py` so the store is built once, on first use.
3. Add `sources` and `grounded` to the state.
4. Write `agents/retriever_agent.py`: `route_search`, `is_relevant` (best score ≤ `MAX_DISTANCE`), `retrieve_node`, `has_grounding`, `no_grounding_node`. **Write the bar down before testing.**
5. Write `agents/reasoner_agent.py` with `reason_node`: for now, the best excerpt plus its file name as the `answer`.
6. Wire the two conditional edges in `graph.py` — this is the graph's final shape.
7. Write `tests/unit/test_chunking.py` and `tests/unit/test_routing.py` (one function per rule), and add integration cases 5 (a document question is grounded) and 6 (an unknown topic ends at `no_grounding_node`).

Your files after Step 3:

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── data/
│   └── docs/              (new) 5 documents
├── src/
│   ├── agents/
│   │   ├── reasoner_agent.py  (new) reason_node — quotes the best excerpt
│   │   └── retriever_agent.py (new)
│   ├── rag/
│   │   ├── __init__.py    (new)
│   │   ├── chunking.py    (new, Doc08)
│   │   ├── ingest.py      (new, Doc08)
│   │   ├── retriever.py   (new, Doc08)
│   │   └── store.py       (new) get_collection()
│   ├── graph.py           (changed) 2 conditional edges — final shape
│   └── state.py           (changed) + sources, grounded
└── tests/
    ├── integration/
    │   └── test_live_graph.py (changed) + cases 5-6
    └── unit/
        ├── test_chunking.py   (new)
        └── test_routing.py    (new)
```

Why these files (what and why):

- `agents/reasoner_agent.py`
  - **What:** One node, `reason_node(state)`: for now it answers with the best excerpt, word for word, followed by its file name in brackets.
  - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.
- `agents/retriever_agent.py`
  - **What:** `SEARCH_KEYWORDS`, `MAX_DISTANCE`, and five functions: `route_search(state)` → "search" or "skip"; `is_relevant(sources)` compares the smallest score with `MAX_DISTANCE`; `retrieve_node(state)` searches and stores `sources` and `grounded`; `has_grounding(state)` → "reason" or "no_grounding"; `no_grounding_node(state)` returns an honest "not found" answer.
  - **Why:** A vector store always returns something. Only a written relevance bar decides whether it's good enough — otherwise the answer would be built on nothing.
- `rag/chunking.py`
  - **What:** Two functions from Doc08: `chunk_by_chars(text, chunk_size)` cuts the text every N characters; `chunk_by_paragraph(text)` splits on blank lines and drops empty pieces.
  - **Why:** Search works on small pieces, so it can find the right paragraph instead of a whole file.
- `rag/ingest.py`
  - **What:** `build_vector_store(doc_folder, chunk_fn, collection_name)`: delete and recreate the Chroma collection, chunk every `.md` / `.txt` file, embed all chunks in **one** `embeddings.create()` call, and `collection.add()` them with ids and metadata.
  - **Why:** It builds the searchable store from `data/docs/`. Deleting first means running it twice never duplicates chunks.
- `rag/retriever.py`
  - **What:** `retrieve(collection, query, k=3)`: return `[]` for an empty store; otherwise embed the query, `collection.query(...)`, and return a list of dicts with `text`, `source_file`, `chunk_index` and `score` (a distance — lower is closer).
  - **Why:** This is the search itself, returning a checked, simple shape instead of the store's raw output.
- `rag/store.py`
  - **What:** A `StoreHolder` class with a `collection` field, one `store` object, and `get_collection()`, which builds the store only when `store.collection` is still `None`.
  - **Why:** The documents are embedded once per run — not for every question, and not at import time before the API key is loaded.
- `graph.py`
  - **What:** `build_graph(checkpointer)` in its final shape: `clean_task`, then `route_search` picks `retrieve_node` or `answer_node`; after the search,

`has_grounding` picks `reason_node` (then `approval_node`) or `no_grounding_node`.
    - **Why:** The whole flow is in one place you can draw. The app passes a SQLite checkpointer and the tests pass a `MemorySaver`, with no code change.
- `state.py`
    - **What:** Step 2's `DocState` plus `sources` (the found chunks) and `grounded`, both set in `start_state()`.
    - **Why:** All nodes agree on the field names, `path` collects a log of every node that ran, and a run never fails because a field was missing.
- `test_live_graph.py`
    - **What:** Step 2's cases, plus case 5 (a document question is searched and grounded) and case 6 (an unknown topic ends at `no_grounding_node`).
    - **Why:** It proves every path through the graph works with the real model, including both human decisions.
- `test_chunking.py`
    - **What:** Two test functions, each printing `OK` or `WRONG`: 650 characters in chunks of 300 give lengths `[300, 300, 50]`, and `chunk_by_paragraph()` drops the empty pieces between blank lines.
    - **Why:** Search quality starts with the chunks — a chunker bug would quietly make whole paragraphs unfindable.
- `test_routing.py`
    - **What:** Seven test functions, one per rule, on hand-made states and score lists: a document question searches, a general one skips; a best score of 0.4 counts, 1.7 doesn't, no sources at all doesn't; grounded goes to the Reasoner, not grounded to the honest "not found".
    - **Why:** A wrong comparison (`>` instead of `<=`) would only show up as strange answers — here it shows up as `WRONG`.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 1 (650 chars, size 300)** — `tests/unit/test_chunking.py`
    - **Check:** `chunk_by_chars("a" * 650, chunk_size=300)` gives chunks of 300, 300 and 50 characters.
    - **Why:** nothing may be lost or duplicated when a document is cut.
    - **When:** after any change to `rag/chunking.py`.
    - **If not:** the end of every document could silently disappear from the search.
- **case 2 (empty pieces dropped)** — `tests/unit/test_chunking.py`
    - **Check:** `chunk_by_paragraph()` on text with extra blank lines returns only the two real paragraphs.
    - **Why:** an empty chunk would be embedded and could come back as a "match".
    - **When:** same as case 1.
    - **If not:** blank chunks would waste embeddings and pollute search results.

- **cases 1-2 (search or skip)** — `tests/unit/test_routing.py`
  - **Check:** "What is the refund policy?" → `"search"`; "What is the capital of France?" → `"skip"`.
  - **Why:** only document questions should pay for a search.
  - **When:** after any change to `SEARCH_KEYWORDS` or `route_search()`.
  - **If not:** general questions would be searched — and probably answered "not found".
- **cases 3-5 (the relevance bar)** — `tests/unit/test_routing.py`
  - **Check:** a best score of 0.4 counts as relevant; 1.7 doesn't; no sources at all doesn't.
  - **Why:** the score is a distance — the best match is the *smallest* — and this is the line between an answer and an honest "not found".
  - **When:** after any change to `is_relevant()` or `MAX_DISTANCE`.
  - **If not:** a flipped comparison (`>` instead of `<=`) would answer from unrelated text and refuse real matches.
- **cases 6-7 (where grounding leads)** — `tests/unit/test_routing.py`
  - **Check:** `grounded=True` → `"reason"`; `grounded=False` → `"no_grounding"`.
  - **Why:** the edge after the search must follow the judgment, not ignore it.
  - **When:** after any change to `has_grounding()` or the edge map in `graph.py`.
  - **If not:** an ungrounded search could still reach the Reasoner.
- **case 5 (grounded)** — `tests/integration/test_live_graph.py`
  - **Check:** "What is the refund policy?" goes through `retrieve_node` and ends with `grounded=True`.
  - **Why:** proves the real store, the real embeddings and the relevance bar work together on a question the documents answer.
  - **When:** at the end of the step, and after changing documents, `MAX_DISTANCE` or the RAG files.
  - **If not:** a too-strict bar would make every document question "not found".
- **case 6 (not found)** — `tests/integration/test_live_graph.py`
  - **Check:** "What is the policy on remote work?" ends at `no_grounding_node`.
  - **Why:** this is the project's "never makes one up" promise, checked on every run.
  - **When:** same as case 5.
  - **If not:** a too-loose bar would answer from whatever chunk happened to be closest.

Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 4 of 5

- **Where you are now:** after Step 3 document answers are one quoted excerpt — correct, but not a real answer.
- **What's new in this step:** `reason_node` keeps its name and output, but a Doc05 chain now writes the answer from **all** the excerpts; `format_sources()` and the strict prompt

`prompts/reasoner.txt` are new.

- **What stays the same:** everything else — the graph, the state, the Retriever, `approval_node`, `main.py`.
- **Why:** search quality and answer quality fail for different reasons. A separate Reasoner that may only use the excerpts, and must cite them, keeps the two checkable apart.
- **How:** `format_sources()` puts each excerpt's file name in front of it → a chain with the reasoner prompt → `answer`.
- **Working after this step:** document questions get a written answer with `[file.md]` citations, still approved by you; live case 7 prints `OK`.

Read first: Doc10 Build Task (the `reason_node`), Doc05 — LCEL basics.

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. Write `prompts/reasoner.txt`: only the excerpts; cite each fact's file in brackets; an exact sentence for "not covered".
2. In `agents/reasoner_agent.py`, add `format_sources()` and change the inside of `reason_node` to run the chain — keep its name and its output.
3. Add integration case 7: the refund answer must cite `[refund_policy.md]`.
4. Check several answers **by hand** against the documents.

Your files after Step 4:

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── src/
│   ├── agents/
│   │   └── reasoner_agent.py (changed) + format_sources(), model answer
│   └── prompts/
│       └── reasoner.txt      (new)
└── tests/
    └── integration/
        └── test_live_graph.py (changed) + case 7
```

Why these files (what and why):

- `agents/reasoner_agent.py`
  - **What:** The node `reason_node(state)`, which returns the `answer`. In Step 3 it simply quotes the best excerpt with its file name; in Step 4 it keeps its name and output, and a model chain writes the answer instead, with the new helper `format_sources(sources)` putting each excerpt's file name in front of it.
  - **Why:** Answers come only from your documents, and each fact names its file — which the Approval agent then checks.

- `prompts/reasoner.txt`
  - **What:** This text file holds the reasoner prompt. The code loads it with `load_prompt("reasoner")`.
  - **Why:** The Reasoner's rules change most often, so they live in a file you can edit easily.
- `test_live_graph.py`
  - **What:** Step 3's cases, plus case 7: the refund answer must name `[refund_policy.md]`.
  - **Why:** It proves every path through the graph works with the real model, including both human decisions.

Tests for this step — write them yourself (the solution has the code, to compare):

- **case 7 (cites its file)** — `tests/integration/test_live_graph.py`
  - **Check:** the answer to "What is the refund policy?" contains `[refund_policy.md]`.
  - **Why:** citing the file is this step's promise — and Step 5's checklist relies on it.
  - **When:** at the end of the step, and after changing `prompts/reasoner.txt` or `format_sources()`.
  - **If not:** a prompt change that dropped citations would go unnoticed — and in Step 5 every answer would then need a human. Not tested on purpose: whether the written answer is *true*. That needs reading it against the documents — do it by hand, as the step says.

Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents)

Project: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — Step 5 of 5

- **Where you are now:** after Step 4 you approve **every** document answer by hand — safe, but too slow to use every day.
- **What's new in this step:** the Approval agent's checklist: `cites_a_source()`, `asks_for_an_action()` and `run_checks()`. In `approval_node`, the fixed `reasons` line becomes `run_checks(state)`, and an "everything passed → approve" block is added.
- **What stays the same:** the graph, `main.py` (it already shows the reasons), the Retriever, the Reasoner, the pause/resume scripts.
- **Why:** a human check on every run gets skipped in practice; a gate that asks only when a specific check fails is one people keep.
- **How:** `run_checks()` → a list of failed checks (grounded? cites a source? asks for an action?). Empty → auto-approve. Otherwise `interrupt()` with the answer and the failed checks.
- **Working after this step:** the full project — safe answers go straight out, action requests and weak answers wait for you; every test case prints `OK`.

Read first: Doc10 — approval pause, and as a look ahead, Doc11 — Patterns (the Sequential row).

Stuck on this step? Hint 1 · Hint 2 · Show me the solution

What to do:

1. In `agents/approval_agent.py`, add `ACTION_WORDS`, `cites_a_source()`, `asks_for_an_action()` and `run_checks()` — one check per function, so a log line can name the one that failed.
2. In `approval_node`, replace the fixed `reasons` line with `reasons = run_checks(state)`, and add the auto-approve block before the pause.
3. Write `tests/unit/test_approval_checks.py`, and add integration case 8: a safe document question is now **auto-approved**, with no pause. Cases 2-4 (the action task) still pause.
4. Run everything, and the pause/resume scripts once more.

Your files after Step 5 (final): the full tree in Project Structure. What changed:

```
DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely/
├── src/
│   └── agents/
│       └── approval_agent.py (changed) + the checklist, auto-approve
└── tests/
    ├── integration/
    │   └── test_live_graph.py    (changed) + case 8
    └── unit/
        └── test_approval_checks.py (new)
```

Why these files (what and why):

- `agents/approval_agent.py`
  - **What:** The node `approval_node(state)`. From Step 2 it pauses with `interrupt({answer, failed_checks})` and approves only if the person answers "yes". Step 5 adds `ACTION_WORDS` and three check functions (`cites_a_source()`, `asks_for_an_action()`, `run_checks()`), and the node first runs `run_checks()` — if nothing failed, it approves by itself.
  - **Why:** Safe answers go out on their own; anything risky or doubtful waits for a person — who is told exactly which check failed.
- `test_live_graph.py`
  - **What:** Step 4's cases, plus case 8: a safe document question is auto-approved, with no pause.
  - **Why:** It proves every path through the graph works with the real model, including both human decisions.
- `test_approval_checks.py`
  - **What:** Three test functions on hand-made states, each printing `OK` or `WRONG`: a grounded, cited answer passes every check (`[]`); an answer with no citation fails `cites no source file`; a task asking for an action fails `the task asks for an action`.

    - **Why:** A broken check would send every answer to a human, or none — this catches it in a second, with no API key.

Tests for this step — write them yourself (the solution has the code, to compare):

- **cases 1-3 (the checklist)** — `tests/unit/test_approval_checks.py`
    - **Check:** a grounded, cited answer fails nothing (`[]`); no citation fails `cites no source file` ; an action task fails `the task asks for an action` .
    - **Why:** each check must fail for exactly its own reason, so the log can say which one sent an answer to a human.
    - **When:** after any change to `ACTION_WORDS` or a check function.
    - **If not:** a broken check would send every answer to a human — or none.
- **case 8 (auto-approved)** — `tests/integration/test_live_graph.py`
    - **Check:** "What is the refund policy?" with `decision=None` finishes without a pause, with `auto_approved` in the path.
    - **Why:** safe answers going straight out is this step's feature — a tool that stops on every question gets abandoned.
    - **When:** at the end of the step, and after changing any check or the Reasoner's prompt.
    - **If not:** everything would still wait for a person, and nobody would keep using the tool. Cases 2-4 keep passing unchanged: the action task still pauses, because `asks_for_an_action()` catches "send".

Final Deliverable: DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely — a 3-agent LangGraph system in a proper package layout. The Retriever judges its own search. The Reasoner answers only from what was found, and cites it. The Approval agent auto-clears safe drafts and pauses for a human on the rest — and a paused run survives a restart.

Why this really is multi-agent (said plainly): three roles, each a real node with its own job and its own pass/fail judgment. There's no supervisor — the routing is fixed conditional edges, not a dynamic router. That's Project 4.

Run, Test and Configure the Final Project

The same commands and settings as the repo's README.md — once Step 5 is done, this is how the finished project is run, tested and configured.

Configuration

Variable	Required	Default	Meaning
`OPENAI_API_KEY`	yes	—	Your OpenAI API key
`LOG_LEVEL`	no	`INFO`	`DEBUG` , `INFO` , `WARNING` or `ERROR`

Other settings live in code, each in one place:

Setting	Where	Default
Chat model	`models/llm.py` → `MODEL_NAME`	`gpt-4o-mini`
Embedding model	`rag/ingest.py` → `EMBEDDING_MODEL`	`text-embedding-3-small`
Relevance bar	`agents/retriever_agent.py` → `MAX_DISTANCE`	`1.2` (lower = stricter)
Search topics	`agents/retriever_agent.py` → `SEARCH_KEYWORDS`	policy, refund, shipping, ...
Action words	`agents/approval_agent.py` → `ACTION_WORDS`	send, cancel, delete, ...

Your own documents: put `.md` or `.txt` files in `data/docs/` — plain paragraphs, no heading lines — and update `SEARCH_KEYWORDS` to match their topics. The store is rebuilt from the folder at startup.

Usage

Run from the repo root:

```
python main.py
```

To see a paused run finished by a separate process:

```
python -m scripts.pause_demo     # starts a run that needs approval, exits
python -m scripts.resume_demo    # a new process finds it and finishes it
```

Running the Tests

Run from the repo root:

```
# unit tests — no API key, no network, under a second
python -m tests.unit.test_approval_checks
python -m tests.unit.test_chunking
python -m tests.unit.test_config
python -m tests.unit.test_nodes
python -m tests.unit.test_routing

# integration tests — real API calls, needs OPENAI_API_KEY
python -m tests.integration.test_live_graph
```

Every test case prints `OK` or `WRONG`. The unit tests check the config, the two small nodes, the chunkers, every routing and relevance rule, and the approval checklist. The integration test runs every path: a general question, an action request that pauses — approved in one run, rejected in another — a grounded answer, a question the documents don't cover, a cited answer, and a safe answer that is auto-approved.

Checklist Before You Call This Done

- [ ] The repo folder name matches the GitHub repo name exactly
- [ ] `main.py` at the repo root, the code in `src/`; runs with `python main.py`
- [ ] Every folder has one clear job, and you can say why it exists
- [ ] A `StateGraph` with typed state and at least one real conditional edge
- [ ] A SQLite checkpointer — a paused run is finished by a **separate** process (`python -m scripts.pause_demo` → `python -m scripts.resume_demo`)
- [ ] The Retriever runs only when needed, and judges the best score against a written bar
- [ ] A question the documents don't cover takes the "not found" path
- [ ] The Reasoner answers only from the found excerpts, and cites the file
- [ ] The Approval agent approves safe answers by itself, and asks a human only when a check fails — both paths tested
- [ ] Every test case prints `OK` — `tests/unit/` with no API key, `tests/integration/` with one
- [ ] You wrote each step's tests yourself, and can say for each one what goes wrong without it
- [ ] `.env` and `checkpoints.sqlite` are not in git
- [ ] You can draw the graph from the code without running it

Publish It to GitHub

1. Copy README.md from this folder into your repo folder, and put your own name in its License line.
2. Check nothing secret is staged, then commit:

```
git status    # .env, .venv/, checkpoints.sqlite must NOT appear
git add .
git commit -m "DocuMind: 3-agent LangGraph RAG with human approval"
```

3. On GitHub, create a new repo named exactly `DocuMind-Smart-Document-Agent-Ask-Your-Documents-Anything-Safely` (no README — you have one). In the repo's **Description** box, paste the **Short description** line from the top of README.md. Then:

```
git branch -M main
git remote add origin <your-repo-url>
git push -u origin main
```

Status

Not started.

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**